GİRİŞ

Eğitmen

Elif ÖZKAN



linkedin.com/in/eliffozkan/
github.com/eliffozkan

Özgür YILDIRIM



linkedin.com/in/Ozgur-yldrm/
github.com/OzgurYldrm

- Ders 1: Lineer regresyon
- MSE / RMSE / MAE
- Vectorized gradient descent
- Ders 2: Lojistik regresyon
- Sigmoid, BCE ve karar sınırı

Sunum + Notebook + Homework → Github

Input-Output

Klasik kodlama: Girdi + yazdığımız kurallar → çıktı





Supervised: Etiketli örneklerle öğren → yeni örneği tahmin et





Unsupervised: Etiketsiz veride örüntü / gruplar bul






Reinforcement: Eylem → ödül → davranışı iyileştir



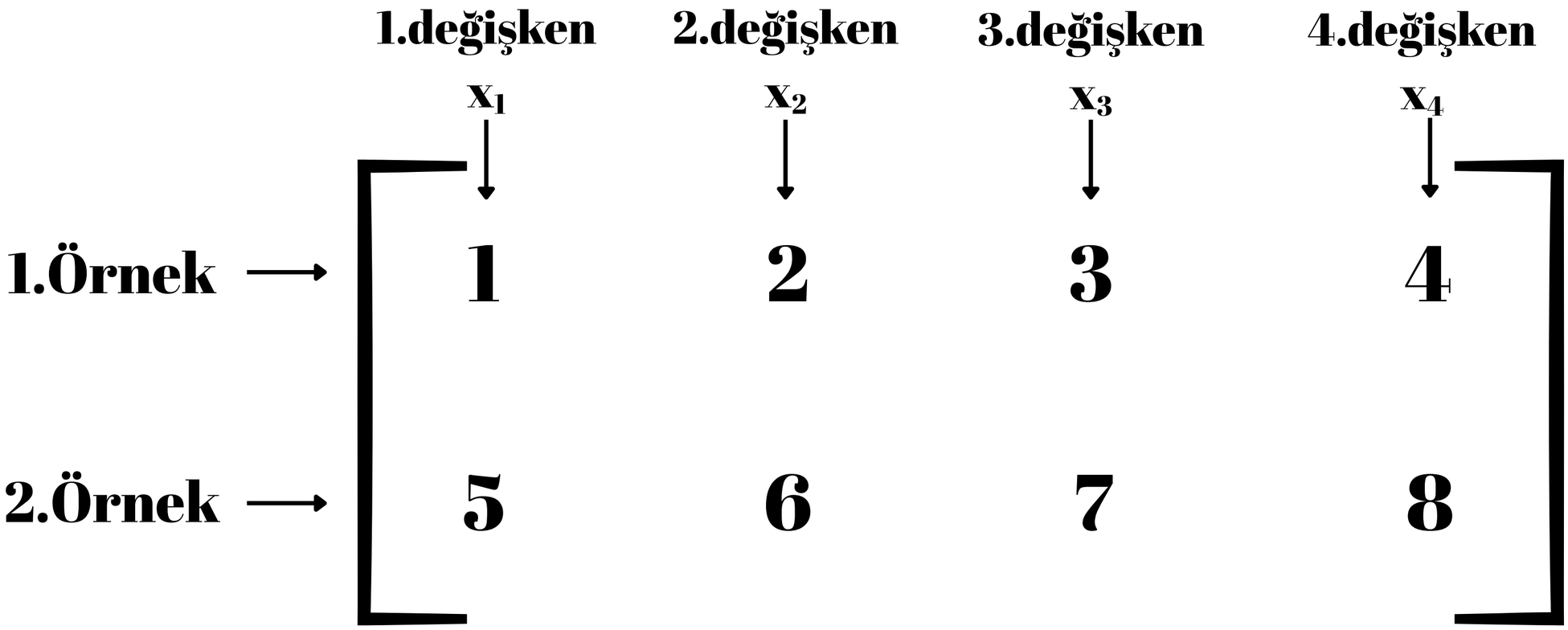


Input-Output Türleri

INPUT (Features)	ÖRNEK
Integer	3, 5, -7
Float	3.14 , 0.21
Boolean	Sosyal sigorta? , Çalışıyor mu? , Evli mi?
Class	Cinsiyet? , Ehliyet tipi? , kanser tipi?
Metin, Resim, Ses ...	Sayı ile ifade çabası

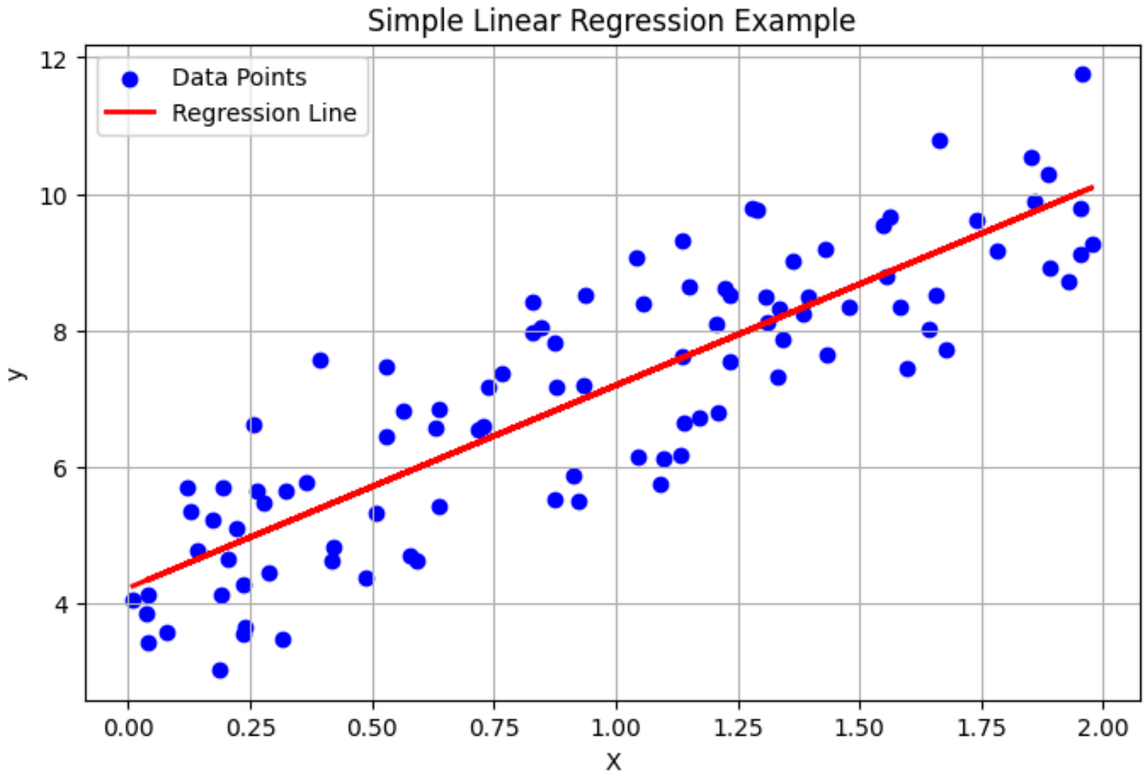
OUTPUT	ÖRNEK
Continuous	0.21 , 8.56
Class	0: köpek 1: kedi 2: kuş
Rank	8: en iyi sigorta 0: en kötü sigorta
Metin, Resim, Ses	Sayısal çıktıların dönüştürülmesi ile oluşur.

Matrix gösterim



Satır = örnek, sütun = özellik. Bu dönem gradient descent için tek vectorized uygulama kullanıyoruz.

Linear Regression Amacı

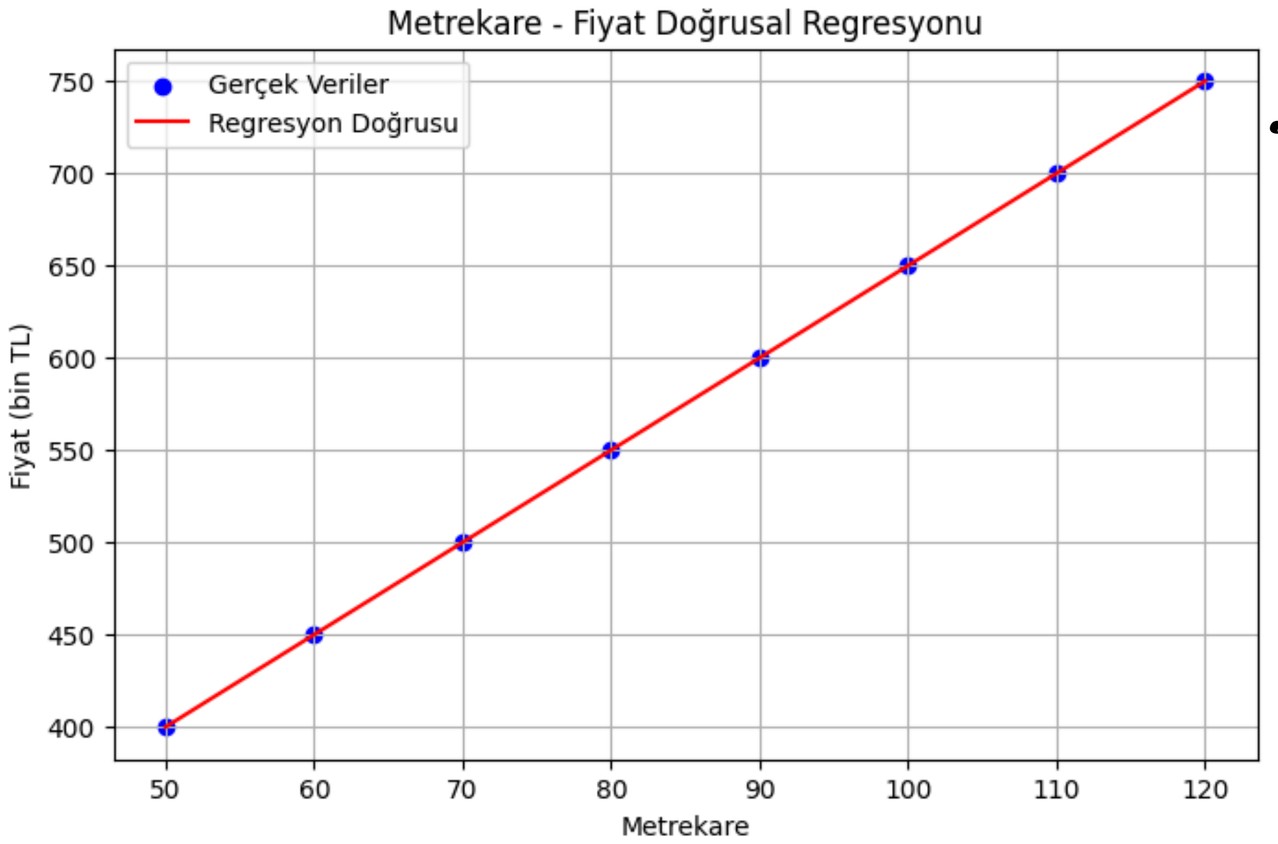


- **Sürekli çıktı üreterek en yakın tahmini yapmak (Function Approximation)**
  - **Ev fiyatı tahmini**
  - **Proje bütçe hesaplaması ...**

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Linear Regression Amacı

Metrekare (X)	Fiyat (Y, bin TL)
50	400
60	450
70	500
80	550
100	650



$y = 5x + 150$

Parametre

- **Eğitilmiş modeli kullarak farklı evler için tahmin yapabiliriz.**

120 m^2 evin tahmini fiyatı (y') → 5.(120) + 150 = 750

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Linear Regression Sorular

- Parametreleri nasıl buluruz?
- Sadece tek bir değişken olmak zorunda değil?
- Algoritmanın ne kadar doğru çalıştığını nasıl bileceğiz?
- Tamamen doğru sonuç vermiyor :(

Loss Functions

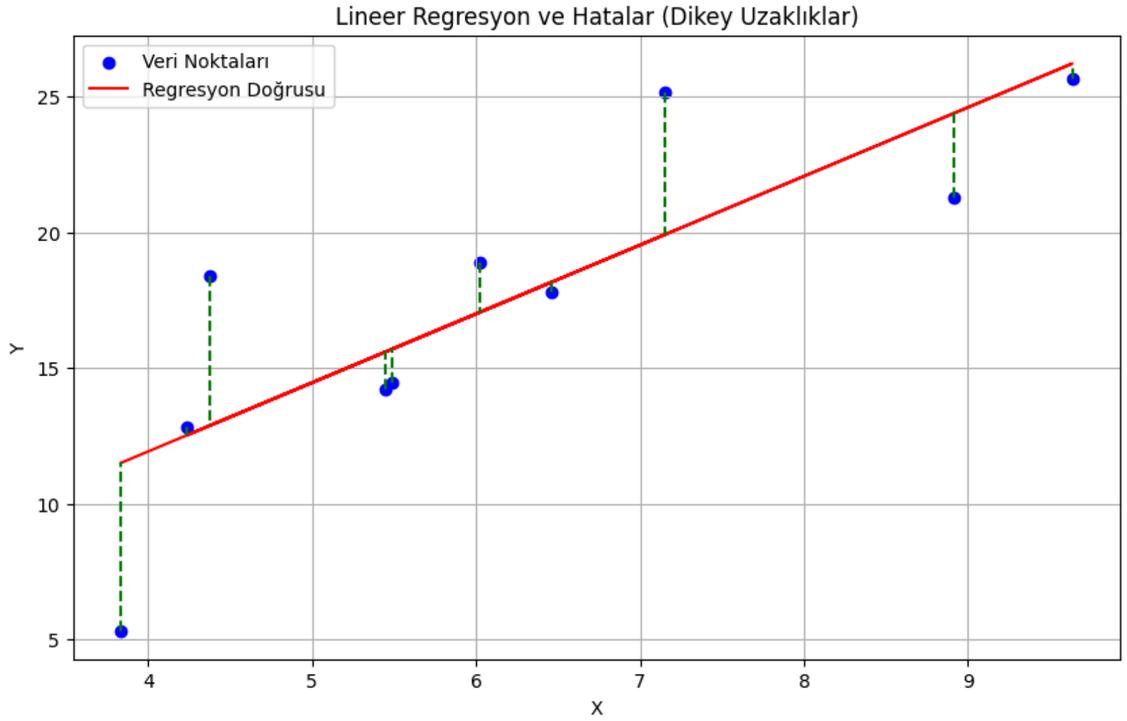
Ground Truth (y)	Prediction (y')
10	12
20	18
30	30

Ground Truth: Gerçek hedef değer
Prediction: Modelin tahmini
Residual: Gerçek değer – tahmin
Loss function: Tahmin hatasını hesaplayan fonksiyon

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Mean Squared Error

Y_i : i. gerçek değer
Y'_i : i. tahmin
n: örnek sayısı



Residual

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

MSE
Mean Squared Error

MSE

Ground Truth (y)	Prediction (y')	Loss
10	12	4
20	18	4
30	30	0

Loss = 8/3

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(Y_i - \hat{Y}_i)^2$$

Kare olmazsa

Ground Truth (y)	Prediction (y')	Loss
10	12	-2
20	18	2
30	30	0

Loss = 0

***Loss genellikle negatif olmaz**

- **Kare alma, büyük hataları daha fazla cezalandırır.**

RMSE
Root Mean Squared Error

RMSE

Ground Truth (y)	Prediction (y')	Loss
10	12	4
20	18	4
30	30	0

$$\text{Loss} = (8/3)^{1/2}$$

$$\text{RMSE} = \sqrt{\frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}}$$

MAE
Mean Absolute Error

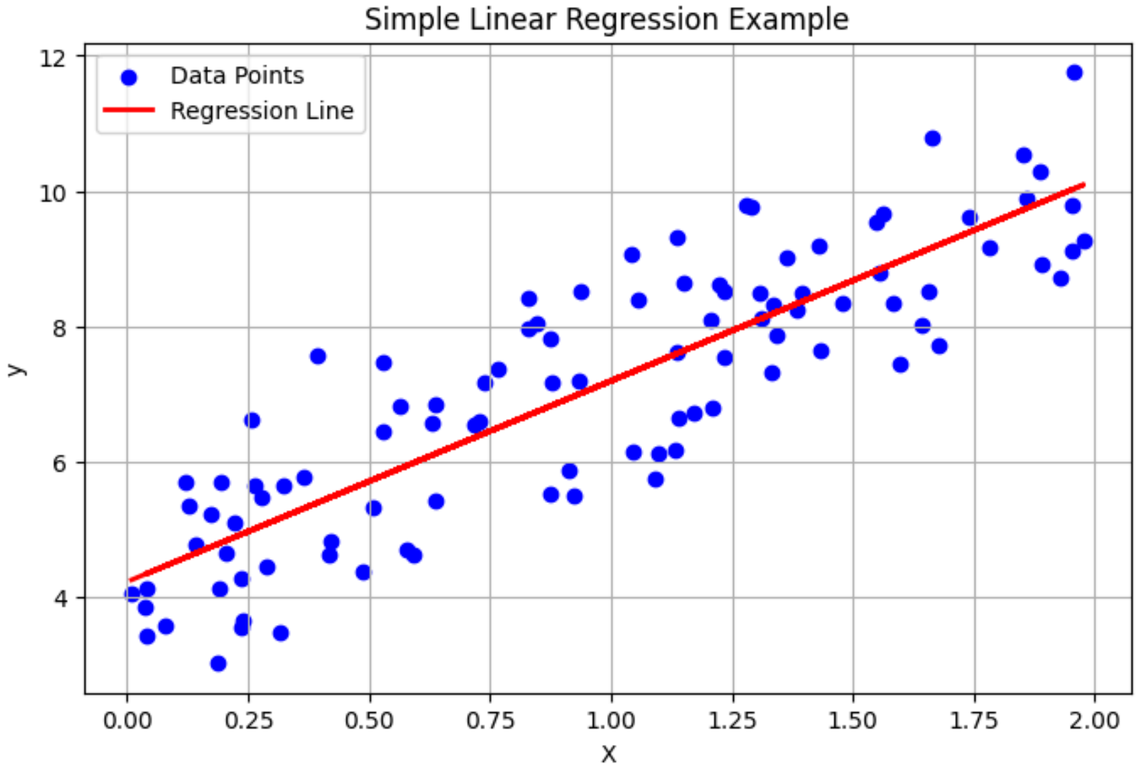
Ground Truth (y)	Prediction (y')	Loss
10	12	2
20	18	2
30	30	0

Loss = 4/3

$$MAE = \frac{\sum_{i=1}^{n} |y_i - \hat{y}_i|}{n}$$

RMSE ve MAE hedefle aynı birimdedir; MSE birimi hedefin karesidir.
Huber / Smooth L1: büyük hataların etkisini sınırlayan alternatifler.

Linear Regression Model



$$y = wx + b$$

w: weight

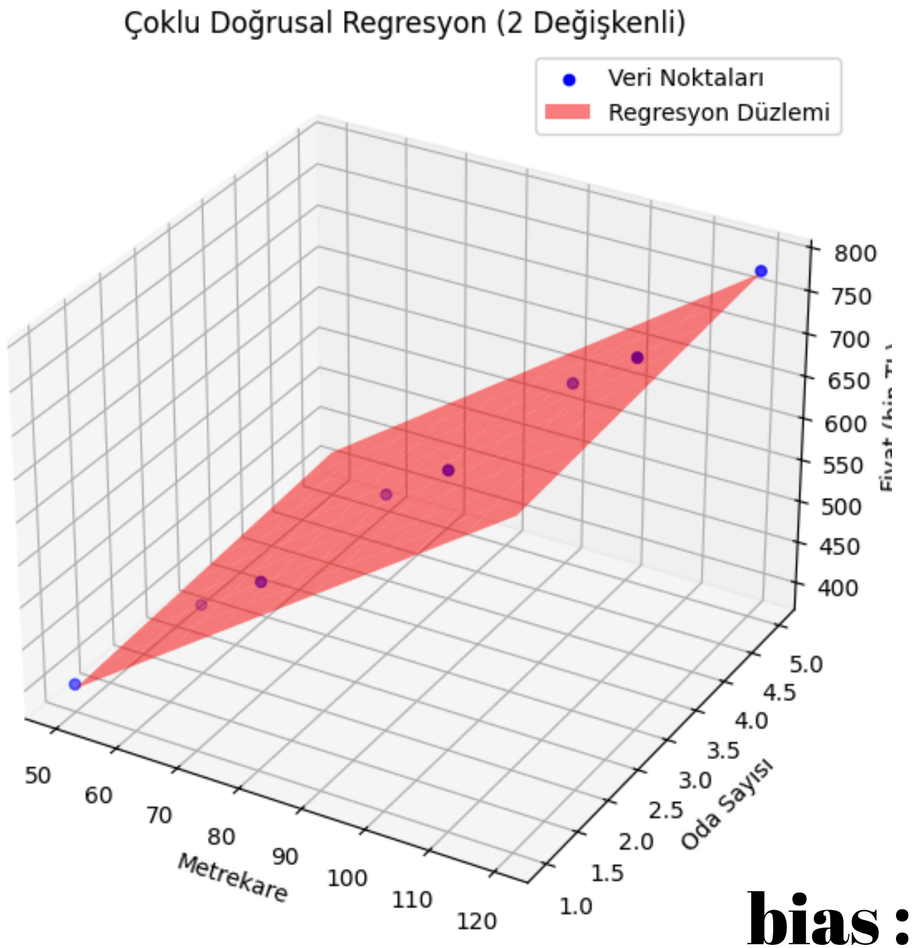
b: bias

Amaç en iyi w ve b değerlerini bulmak

Amaç en iyi çizgiyi çekmek

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Multivariable Linear Regression



Genel formül

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

w_i : i. weight

x_i : i. özellik

b : bias

bias : 133

w_1: 5.1

w_2: 6.5

$\longrightarrow$ X_1 : 75, X_2 : 2 $\longrightarrow$ (5.1 * 75) + (6.5 * 2) + 133

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Multivariable Linear Regression Matrix gösterimi

$$w_1 = 1,\quad w_2 = 2,\quad w_3 = 3 \Rightarrow w = \begin{bmatrix} 1 \to w_1 \\ 2 \to w_2 \\ 3 \to w_3 \end{bmatrix}$$

3×1

$$* \quad y = w_1 x_1 + w_2 x_2 + w_3 x_3 + \cdots + w_n x_n$$

$$x = \begin{bmatrix} x_1 & x_2 & x_3 \\ 1 & 2 & 3 \\ 4 & 5 & 6 \\ -1 & -2 & -3 \\ -4 & -5 & -6 \end{bmatrix}$$

4×3

$$x \cdot w = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ -1 & -2 & -3 \\ -4 & -5 & -6 \end{bmatrix} \cdot \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} = \begin{bmatrix} 14 \\ 32 \\ -14 \\ -32 \end{bmatrix}$$

$4 \times 3 \quad 3 \times 1 \quad 4 \times 1$

***Hint: np.matmul()**

Gradient Descent

Gradient Descent

Repeat until converge {

$$w = w - \alpha \left[\frac{\partial Loss}{\partial w}\right]$$

$$b = b - \alpha \left[\frac{\partial Loss}{\partial b}\right]$$

}

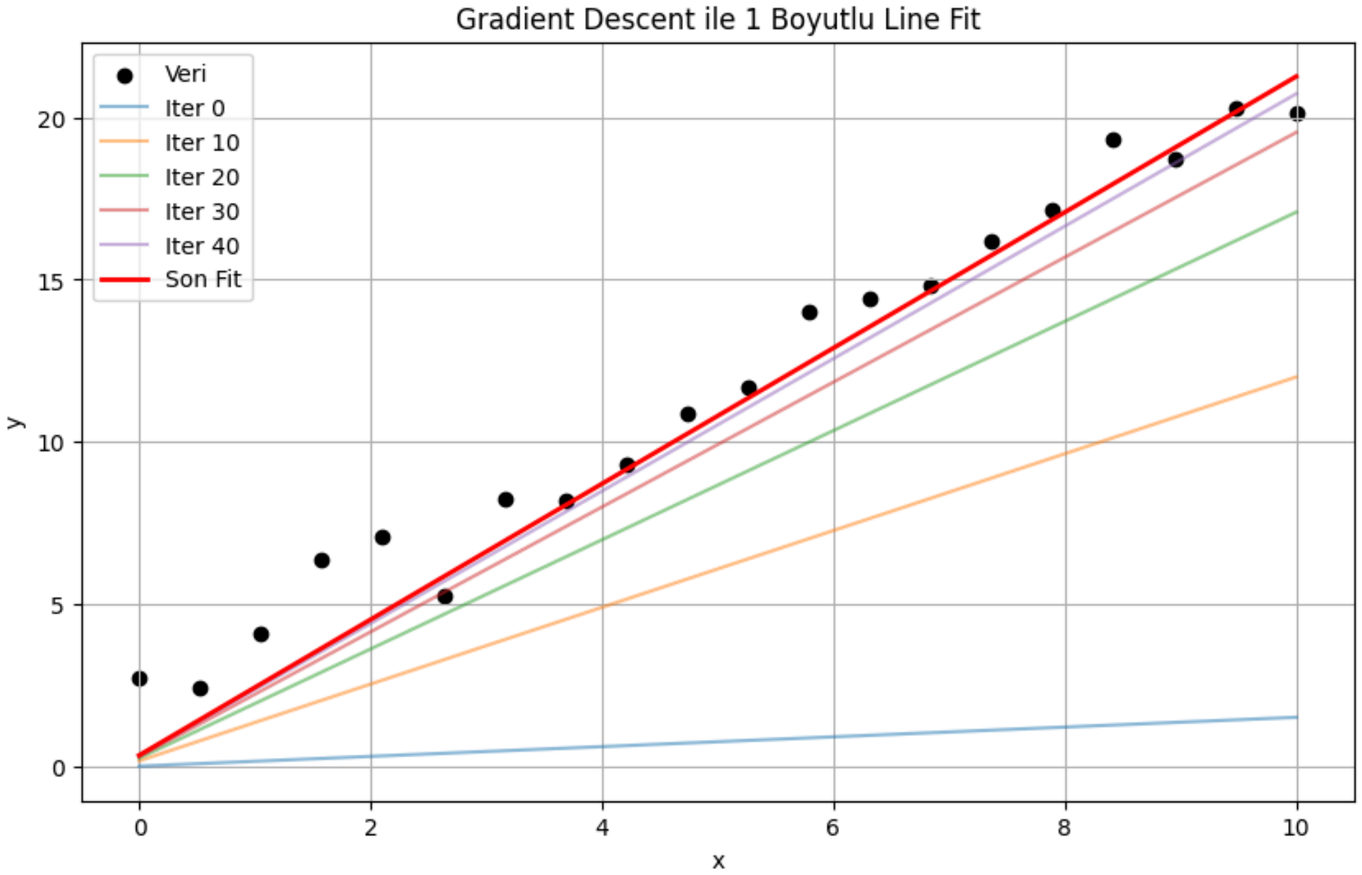
w: weight

b: bias

alpha: Learning rate

Partial Derivation

Gradient Descent Nasıl görünüyor



Repeat until converge { -> **ne demek**

converge:Hatanın minimuma yaklaşması

iteration: weight-bias güncellemelerinin yapıldığı her bi adım

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Partial Derivative

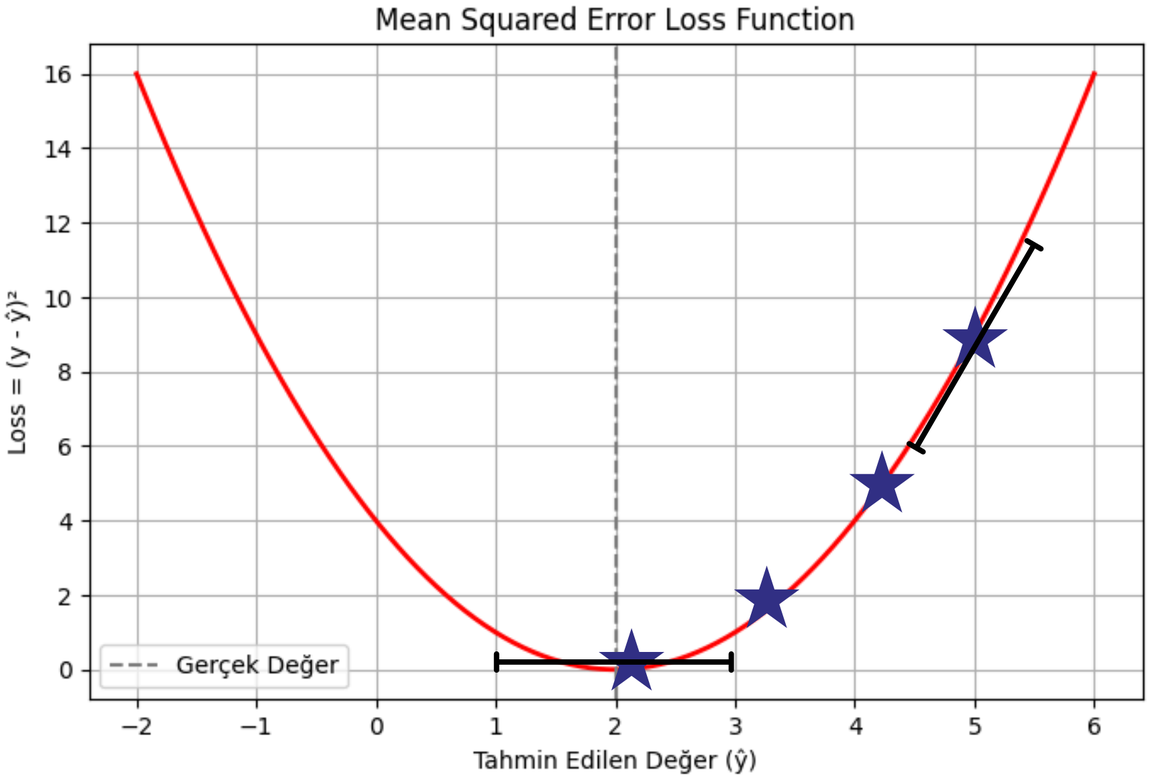
$$MSE = \frac{1}{n}\sum (y-\hat{y})^2 \qquad (\hat{y} = w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_m \cdot x_m + b)$$

$$MSE = \frac{1}{n}\sum (y - [w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_m \cdot x_m + b])^2$$

$$\frac{\partial (MSE)}{\partial w_i} = \frac{-2}{n}\sum (y - [w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_m \cdot x_m + b]) \cdot x_i$$

$$\frac{\partial (MSE)}{\partial b} = \frac{-2}{n}\sum (y - [w_1 \cdot x_1 + w_2 \cdot x_2 + \cdots + w_m \cdot x_m + b])$$

Gradient Descent



istenilen değer

İstenilen fonksiyon → y = 2x

Adım	w	y - (wx)	2α × (wx − y) × x
1	5	-3	0.3
2	4.7	-2.7	0.27
n	2.01	-0.01	0.001

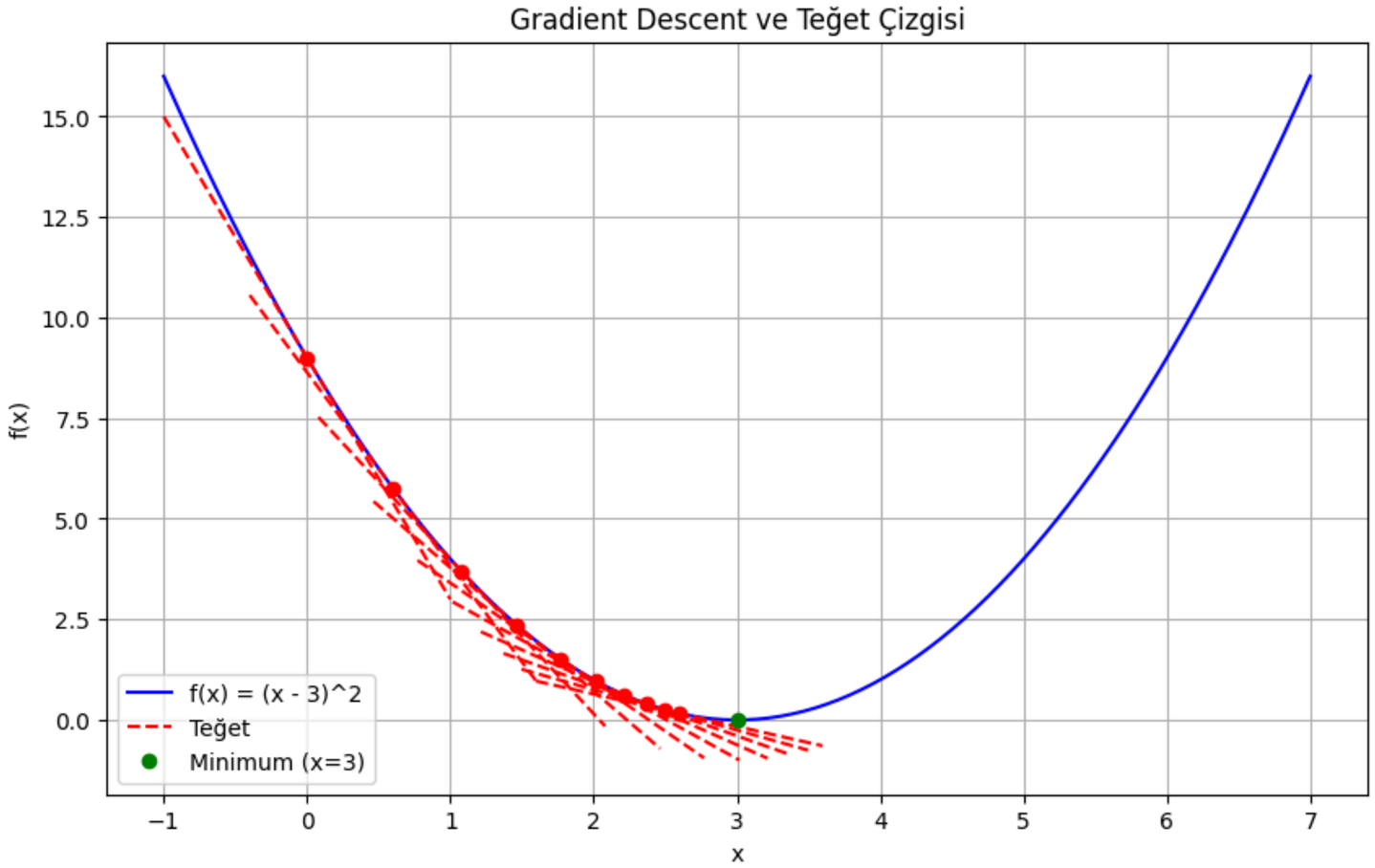
x=1 y=2 α=0.05; b=0 sabit

$$w = w - \alpha \left[\frac{\partial Loss}{\partial w}\right]$$

$$b = b - \alpha \left[\frac{\partial Loss}{\partial b}\right]$$

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Türevin önemi



1. Güncelleme yönü
2. Güncelleme büyüklüğü
3. Minimumda 0 olma işi

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Gradient Descent Matrix gösterimi

$X = [[1, 2, 3], [4, 5, 6]]$ $y = [14, 32]$ $w = [2, 2, 2]$

Bu örnekte $b = 0$ sabit; $n = 2$, öğrenme oranı $\alpha = 0.01$.

Tahmin: $X @ w = [12, 30]$ Hata: tahmin $- y = [-2, -2]$

MSE $= (4 + 4) / 2 = 4.00$

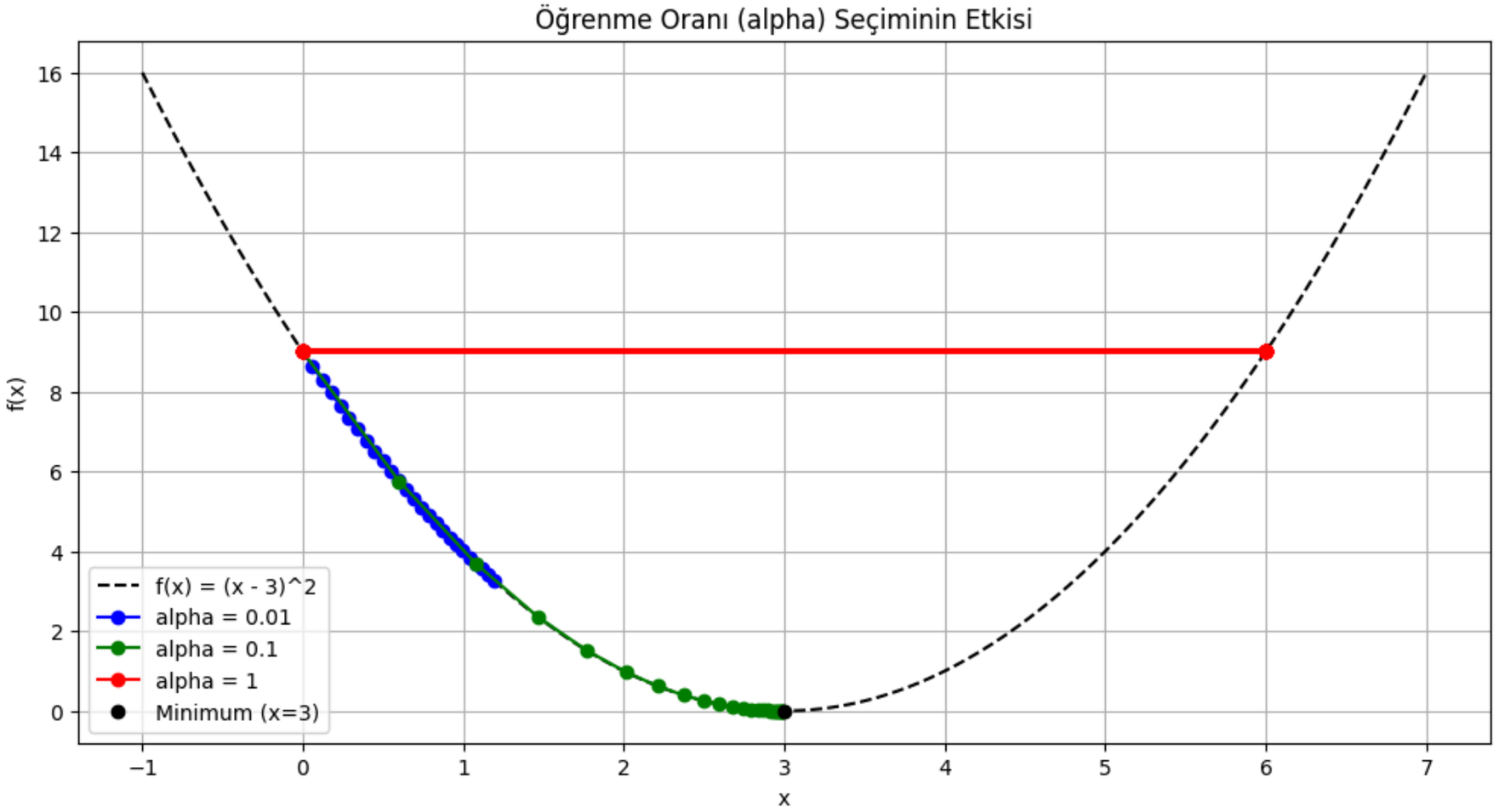
Gradyan: $(2/n)\ X^T @ \text{hata} = [-10, -14, -18]$

Yeni $w = w - \alpha \times \text{gradyan} = [2.10, 2.14, 2.18]$

Yeni tahmin $= [12.92, 32.18]$ Yeni MSE ≈ 0.60

Notebookta w ile birlikte bias da güncellenir.

Learning Rate Seçimi



$$w = w - \alpha \left[\frac{\partial Loss}{\partial w}\right]$$

$$b = b - \alpha \left[\frac{\partial Loss}{\partial b}\right]$$

Her bir güncelleme adımının büyüklüğünü belirler.
büyük alpha → büyük adım

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

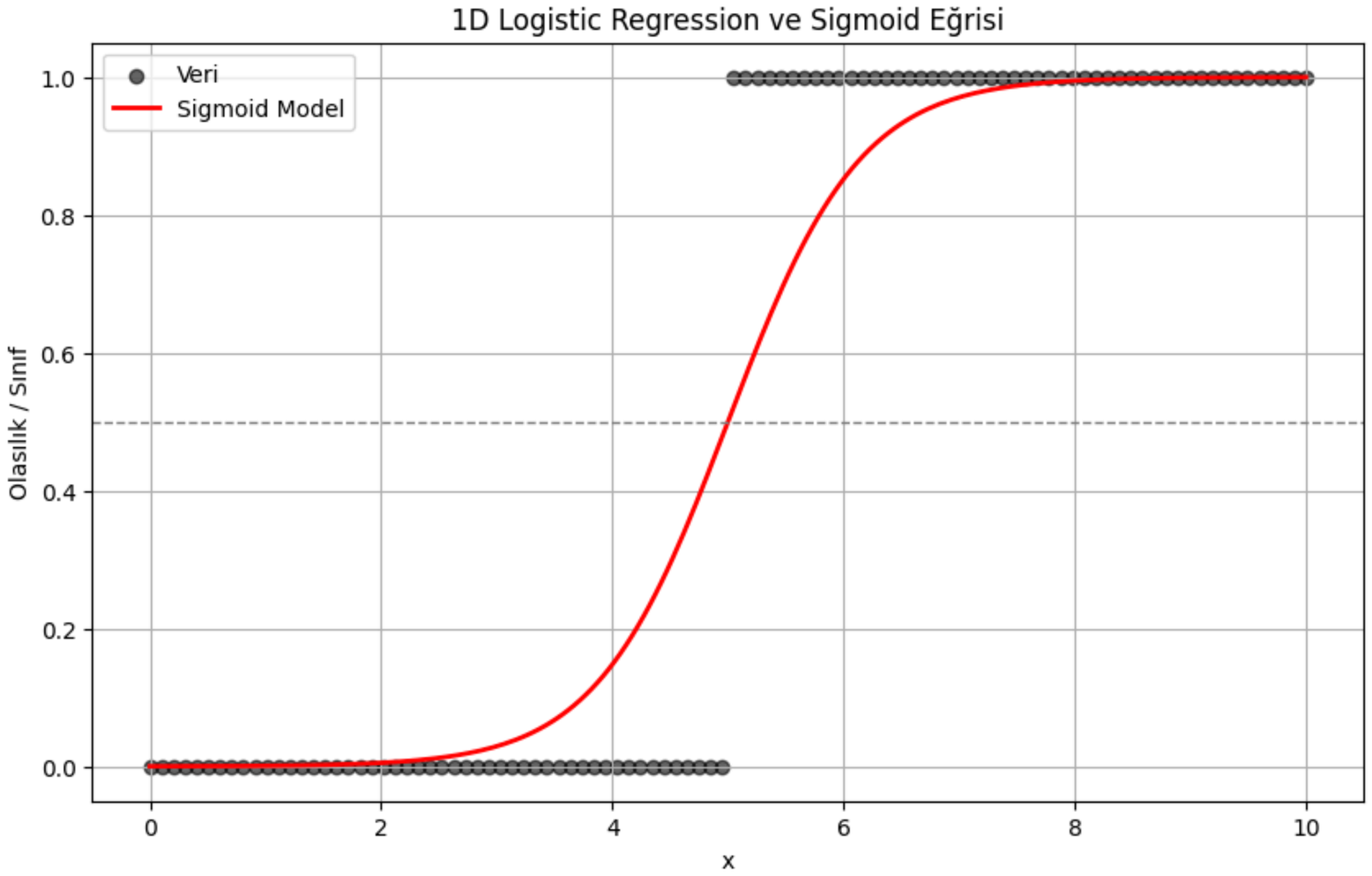
Normal Equation

Normal equation: lineer regresyon için kapalı form alternatifidir. Bu dönem uygulama kodunu yazmıyoruz.

$$w = (x^T \cdot x)^{-1} \cdot x^T \cdot y$$

- **Büyük matrix'ler için hesaplaması pahalı**
- **Tersi alınmayan matrix olma ihtimali var**

Logistic Regression



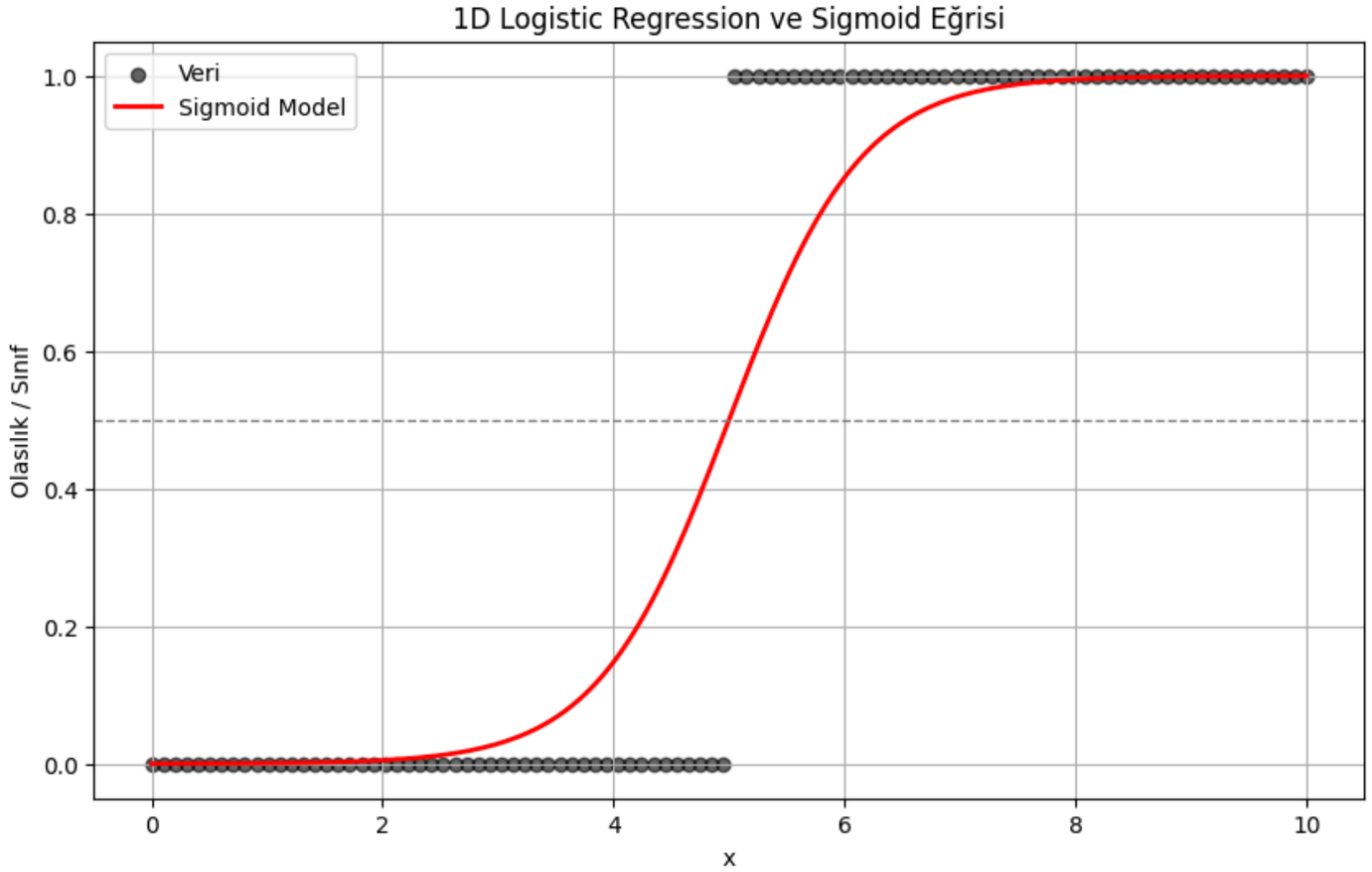
Sınıfsal çıktı üreterek verilen girdilere göre sınıflandırma yapmak

- **Müşterinin yaş ve gelirine göre aboneliği bırakma tahmini**

Genellikle
1 → pozitif sınıf
0 → negatif sınıf

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Logistic Regression amacı



x=7 inputu için → %90 pozitif sınıf

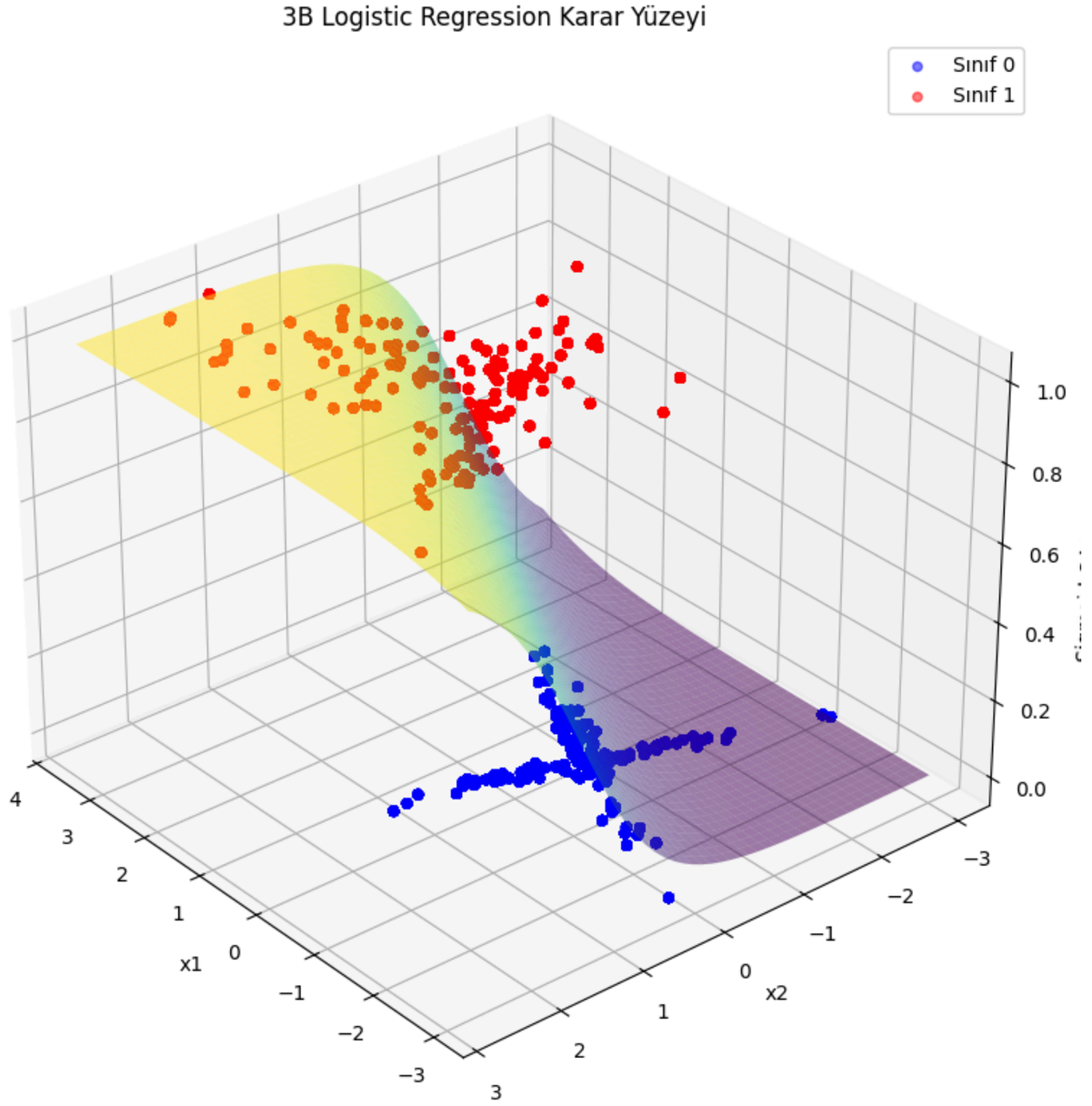
Eğer fonksiyon elimizde olursa yeni girdiler için sınıflandırma yapabiliriz

Gerçek etiketler 0 veya 1; modelin çıktısı sınıf-1 olasılığıdır.

***Multilabel → Belki sonra**

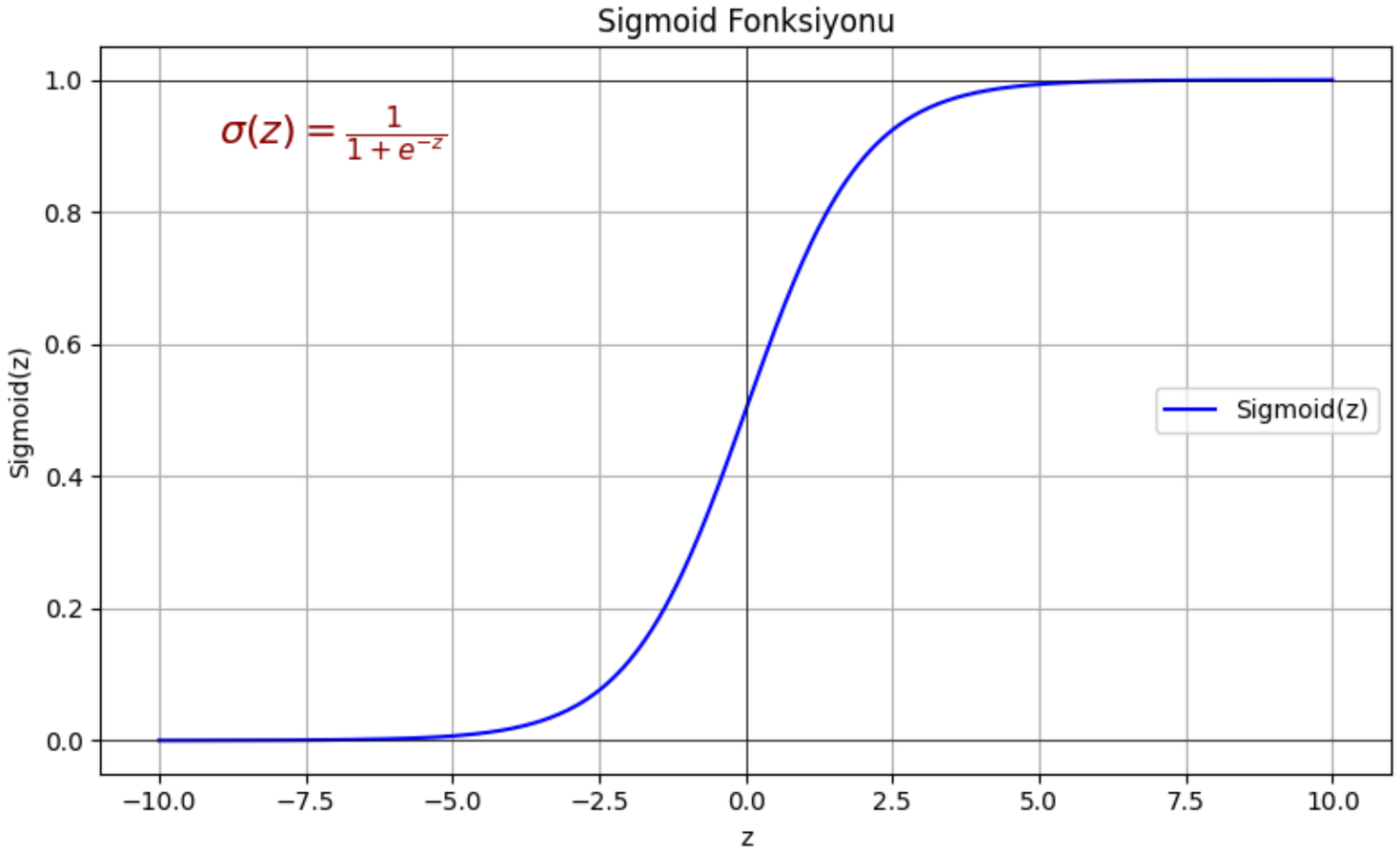
* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Multivariable Logistic Regression nasıl görünüyor



* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Sigmoid



$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

w: weight

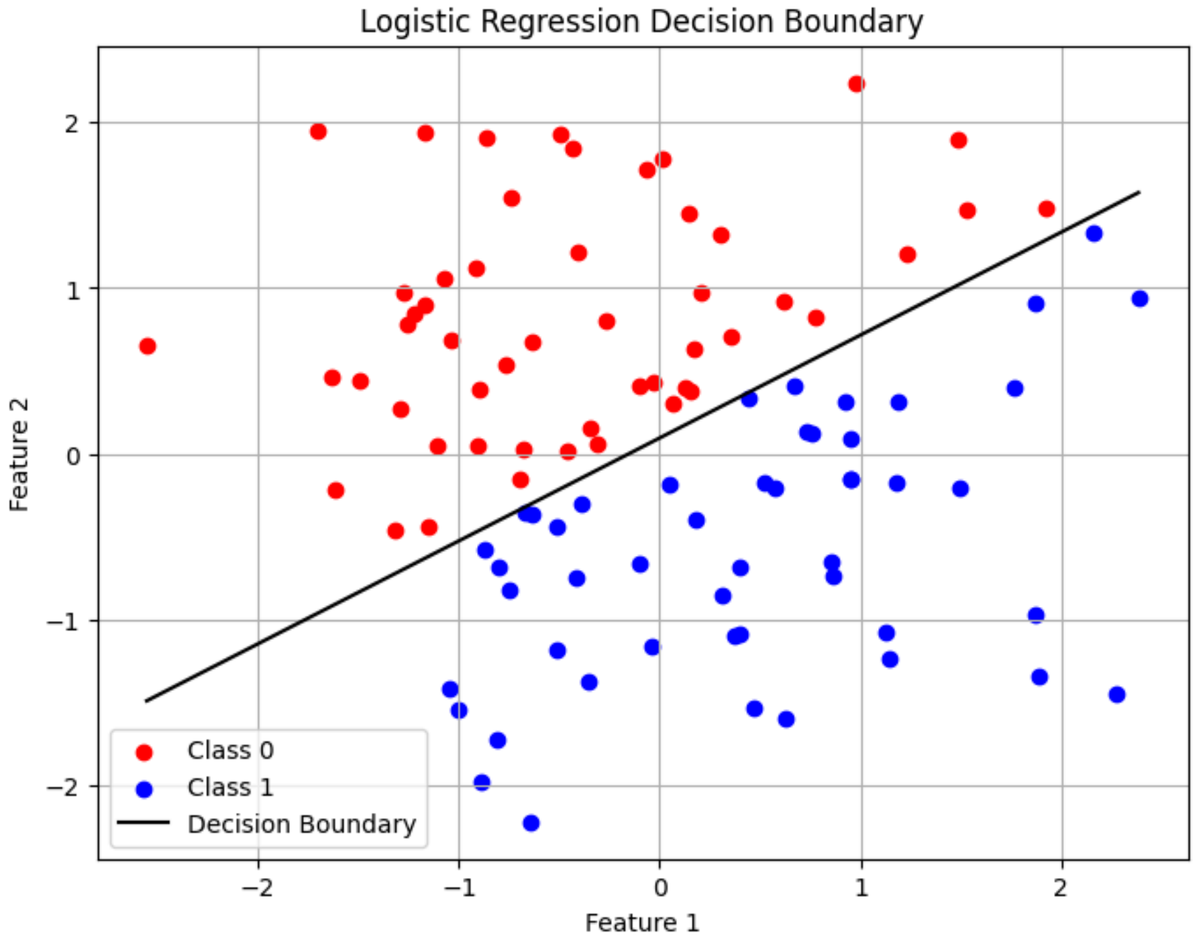
b: bias

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

Her zaman 0 ile 1 arasında çıktı

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Logistic Regression Prediction



$w_1 = 2.8$

$w_2 = -4.5$

$b = 0.4$

New input

$x_1 = 1$

$x_2 = 1.5$

$$z = 2.8 \times 1 - 4.5 \times 1.5 + 0.4 = -3.55$$

$$p = \text{sigmoid}(z) \approx 0.028$$

$$p < 0.5 \rightarrow \text{sınıf } 0$$

Matrix ile tahmin etmek için: sigmoid(x.W + b)

* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Karar Eşiği (Threshold)

Pozitif sınıfları gözden kaçırma ihtimali

> %50



< %50

Pozitif sınıfları daha rahat tespit etme

0.5: yaygın başlangıç eşiği

Eşik, yanlış pozitif / yanlış negatif maliyetine göre seçilir.

Karar sınırı: p=0.5 için X @ w + b = 0.

Eşik seçimini Modül 2'de tekrar ele alacağız.

BCE
Binary Cross Entropy

$$BCE = -\frac{1}{n}\sum_{i=1}^{n}(y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i))$$

y	y'	y log(y')	(1-y) log(1-y')
1	0.9	-0.10	0
1	0.7	-0.35	0
0	0.3	0	-0.35

Loss: -0.80/-3 = 0.27

1.kısım $y_i \cdot \log(\hat{y}_i)$

2.kısım $(1 - y_i) \cdot \log(1 - \hat{y}_i)$

-1/n önemli ! hata pozitif olmalı

iç kısım her zaman negatif (Sigmoid)

Grad Descent for Logistic Regression

Gradient Descent

Repeat until converge {

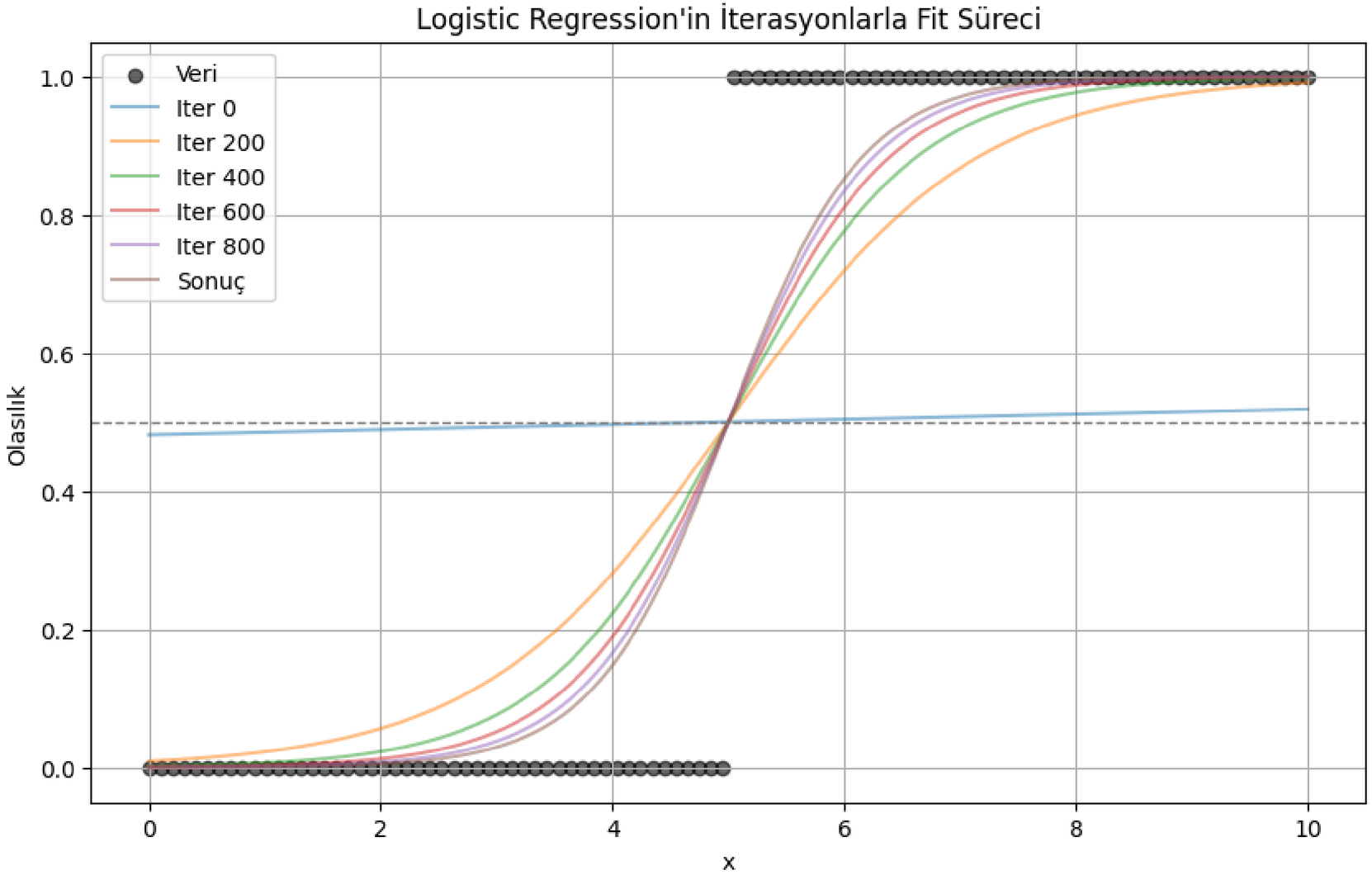
$$w = w - \alpha \left[\frac{\partial Loss}{\partial w}\right]$$

$$b = b - \alpha \left[\frac{\partial Loss}{\partial b}\right]$$

}

***Loss function değişti**

Grad Descent Nasıl görünüyor



* Grafiği üreten python kodu ChatGPT tarafından hazırlanmıştır

Gradient Descent for Logistic Regression

$$w = w - \frac{\alpha}{n} \sum [\sigma(z) - y] \cdot x$$

$$b = b - \frac{\alpha}{n} \sum [\sigma(z) - y]$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

σ: Sigmoid

$z = w_1 \cdot x_1 + \cdots + w_m x_m + b$

Gradient Descent Matrix gösterimi

$X = [[1, 2, 3], [4, 5, 6]]$ $y = [0, 1]$ $w = [2, 2, 2]$

Bu örnekte $b = 0$ sabit; $n = 2$, öğrenme oranı $\alpha = 0.1$.

$z = X @ w = [12, 30]$ $p = \text{sigmoid}(z) \approx [0.999994, 1]$

Gradyan $= X^T @ (p - y) / n \approx [0.5, 1.0, 1.5]$

Yeni $w = w - \alpha \times \text{gradyan} \approx [1.95, 1.90, 1.85]$

BCE önce ≈ 6.00 BCE sonra ≈ 5.65

Hesaplarda tam hassasiyet; yalnızca ekranda yuvarlama.

Notebookta bias da güncellenir; clip, log(0) sorununu önler.

Özet ve Ödev 1

Lineer: fit → predict → MSE (ev fiyatı)

Lojistik: olasılık → eşik → sınıf (müşteri terk)

Ödev 1: iki modeli sklearn ile kur ve self-check çalıştır.

Sonraki modül: yeni veride performansı değerlendirme.